

Capítulo 0: Introducción a React y TypeScript

Esta guía está pensada para desarrolladores que trabajan en FocuseAr y quieren entender los conceptos y herramientas más usados en el proyecto.

¿Qué es React?

React es una biblioteca de JavaScript para construir interfaces de usuario de forma declarativa y basada en componentes. Permite crear aplicaciones web modernas, reutilizando piezas de UI llamadas "componentes".

¿Qué es TypeScript?

TypeScript es un superset de JavaScript que agrega tipado estático. Permite detectar errores antes de ejecutar el código y mejora el autocompletado y la documentación en el editor.

Conceptos clave que más usamos

- **Componentes funcionales:** La unidad básica de UI en React. Son funciones que retornan JSX.
- **JSX:** Sintaxis similar a HTML que se usa dentro de JavaScript/TypeScript para describir la UI.
- **Props:** Son los datos que se pasan de un componente padre a uno hijo.
- **Hooks:** Funciones especiales de React para manejar estado (`useState`), efectos (`useEffect`), contexto (`useContext`), etc.
- **Estado (State):** Información interna de un componente que puede cambiar y hacer que la UI se actualice.
- **Context:** Permite compartir datos globales (como el usuario autenticado) entre componentes sin pasar props manualmente.
- **Tipado con TypeScript:** Definir tipos para props, estados y funciones para evitar errores y mejorar la mantenibilidad.

Herramientas y librerías frecuentes en FocuseAr

- **React Router:** Para navegación entre páginas.
- **Firebase:** Autenticación y base de datos en tiempo real.
- **Tailwind CSS:** Utilidad para estilos rápidos y responsivos.
- **Vite:** Herramienta de build y desarrollo rápido.

Guía de React y TypeScript para FocuseAr

Capítulo 1: Componentes Funcionales y JSX

¿Qué es un componente funcional?

Un componente funcional en React es una función de JavaScript o TypeScript que recibe un objeto de propiedades (props) y retorna lo que se debe mostrar en pantalla usando JSX. Es la forma más simple y recomendada de crear componentes en React moderno.

¿Qué es JSX?

JSX (JavaScript XML) es una extensión de sintaxis que permite escribir código similar a HTML dentro de archivos JavaScript/TypeScript. React transforma este código en llamadas a funciones que crean los elementos de la interfaz.

Ejemplo explicado paso a paso

```
import React from 'react'; // Importa React para poder usar JSX

// Definimos el tipo de las props usando TypeScript
type Props = {
  mensaje: string; // Esta prop será un texto que recibirá el componente
};

// Definimos el componente funcional usando una función flecha
// 'type React.FC<Props>' indica que el componente recibe props tipadas
const MiComponente: React.FC<Props> = ({ mensaje }) => {
  // Retornamos JSX: un div que muestra el mensaje recibido
  return <div>{mensaje}</div>;
};
```

Explicación de cada parte:

- `import React from 'react';`: Necesario para que JSX funcione correctamente (en proyectos modernos con React 17+ puede no ser obligatorio, pero es buena práctica).
- `type Props = { mensaje: string };`: Definimos el tipo de las props que recibirá el componente, usando TypeScript para mayor seguridad.
- `const MiComponente: React.FC<Props> = ...`: Creamos el componente funcional, indicando que recibe props del tipo definido.
- `({ mensaje }) => { ... }`: Usamos destructuring para acceder directamente a la prop `mensaje`.
- `return <div>{mensaje}</div>;`: El componente retorna JSX, que se verá como HTML en la interfaz.

¿Cómo se usa este componente?

```
<MiComponente mensaje="¡Hola equipo FocuseAr!" />
```

Esto mostrará en pantalla: `¡Hola equipo FocuseAr!`

Capítulo 2: Props y Tipado con TypeScript

En React, las "props" (propiedades) son la forma de pasar datos y funciones de un componente padre a un componente hijo. TypeScript permite tipar estas props para que el código sea más seguro y fácil de mantener.

¿Por qué tipar las props?

Al definir el tipo de las props, evitamos errores comunes y obtenemos autocompletado en el editor. Así, si intentamos pasar una prop incorrecta, TypeScript nos avisa.

Ejemplo real del proyecto: componente funcional con props tipadas

Supongamos que tenemos un componente para mostrar un botón personalizado:

```

type BotonProps = {
  texto: string;
  onClick: () => void;
};

const Boton: React.FC<BotonProps> = ({ texto, onClick }) => (
  <button onClick={onClick}>{texto}</button>
);

```

Explicación:

- `type BotonProps = { ... }`: Definimos el tipo de las props que recibirá el componente.
- `texto: string`: El texto que se mostrará en el botón.
- `onClick: () => void`: Una función que se ejecutará al hacer click.
- `React.FC<BotonProps>`: Indicamos que el componente recibe props del tipo definido.

Ejemplo real de useState tipado en el proyecto

En el archivo `Dashboard.tsx` usamos `useState` con tipado para manejar listas y valores:

```

const [subjects, setSubjects] = useState<Subject[]>([]); // Lista de materias
const [subjectName, setSubjectName] = useState<string>(''); // Nombre de la
materia

```

¿Qué es Subject?

`Subject` es un tipo definido en el proyecto que representa una materia. Por ejemplo:

```

type Subject = {
  id: number;
  name: string;
  examDate: string;
  color: string;
  // ...otros campos
};

```

Así, TypeScript sabe que `subjects` siempre será un array de materias y `subjectName` siempre será un string.

Ejemplo de props en un componente real del proyecto

Supongamos que queremos mostrar una materia en un componente hijo:

```

type MateriaCardProps = {
  materia: Subject;
};

const MateriaCard: React.FC<MateriaCardProps> = ({ materia }) => (
  <div style={{ backgroundColor: materia.color }}>
    <h3>{materia.name}</h3>
    <p>Examen: {materia.examDate}</p>
  </div>
);

```

Así, cuando usemos `<MateriaCard materia={unaMateria} />`, TypeScript validará que `unaMateria` tenga la estructura correcta.

Capítulo 3: El Hook useState

`useState` es un hook de React que permite a los componentes funcionales tener y actualizar su propio estado interno. Cada vez que el estado cambia, el componente se vuelve a renderizar mostrando la nueva información.

¿Cómo se usa useState?

Se importa desde React y se utiliza así:

```
import React, { useState } from 'react';  
  
const [valor, setValor] = useState<TIPO>(valorInicial);
```

- `valor`: Es el valor actual del estado.
- `setValor`: Es la función que permite actualizar ese valor.
- `TIPO`: Es el tipo de dato (opcional, pero recomendado con TypeScript).
- `valorInicial`: Es el valor con el que comienza el estado.

Ejemplos reales del proyecto

Manejar el nombre de una materia (string)

```
const [subjectName, setSubjectName] = useState<string>('');  
// subjectName guarda el texto del input, setSubjectName lo actualiza
```

Manejar una lista de materias (array tipado)

```
const [subjects, setSubjects] = useState<Subject[]>([]);  
// subjects es un array de materias, setSubjects agrega o modifica la lista
```

Manejar un estado booleano (ejemplo: cargando un PDF)

```
const [processingPDF, setProcessingPDF] = useState(false);  
// processingPDF indica si se está procesando un PDF, setProcessingPDF cambia el estado
```

Ejemplo de useState en Login.tsx

```
const [email, setEmail] = useState('');  
const [password, setPassword] = useState('');  
// Ambos estados guardan los valores de los inputs del formulario de login
```

¿Cómo se actualiza el estado?

Para cambiar el valor, se usa la función `setValor`:

```
setSubjectName('Matemática'); // Cambia el nombre de la materia  
setSubjects([...subjects, nuevaMateria]); // Agrega una materia a la lista  
setProcessingPDF(true); // Indica que se está procesando un PDF
```

Cada vez que se llama a la función `set`, el componente se vuelve a renderizar mostrando el nuevo valor.

Capítulo 4: El Hook useEffect

useEffect es un hook de React que permite ejecutar efectos secundarios (side effects) en los componentes funcionales. Se usa para tareas como: llamadas a APIs, suscripciones, sincronización con servicios externos, o manipulación de datos fuera del renderizado.

¿Cómo se usa useEffect?

```
useEffect(() => {  
  // Código a ejecutar después de que el componente se renderiza  
  return () => {  
    // Código de limpieza (opcional)  
  };  
}, [dependencias]);
```

- El primer argumento es una función que se ejecuta después del renderizado.
- El segundo argumento es un array de dependencias: si alguna cambia, el efecto se vuelve a ejecutar.
- Si el array está vacío (`[]`), el efecto solo se ejecuta una vez (al montar el componente).

Ejemplo real del proyecto: sincronización de usuario con Firebase

En `AuthContext.tsx` usamos `useEffect` para escuchar el estado de autenticación y sincronizar el usuario con la base de datos:

```
useEffect(() => {  
  const unsubscribe = onAuthStateChanged(auth, async (firebaseUser) => {  
    if (firebaseUser) {  
      try {  
        // Cuando el usuario se autentica, crear/actualizar en Firestore  
        await DatabaseService.createOrUpdateUser(firebaseUser);  
      } catch (dbError) {  
        // Manejo de error  
      }  
    }  
    setUser(firebaseUser);  
    setLoading(false);  
  });  
  return unsubscribe; // Limpia la suscripción al desmontar  
}, []);
```

Explicación:

- Se ejecuta una vez al montar el componente (array de dependencias vacío).
- Se suscribe a los cambios de autenticación de Firebase.
- Si hay usuario, lo sincroniza con Firestore.
- Al desmontar el componente, se limpia la suscripción llamando a `unsubscribe`.

Ejemplo clásico: temporizador

```
const [segundos, setSegundos] = useState<number>(0);  
  
useEffect(() => {  
  const timer = setInterval(() => setSegundos(s => s + 1), 1000);  
  return () => clearInterval(timer); // Limpia el intervalo al desmontar  
}, []);
```

Capítulo 5: Buenas Prácticas en FocuseAr

En FocuseAr seguimos una serie de convenciones y buenas prácticas para mantener el código ordenado, escalable y fácil de entender para todo el equipo.

1. Uso de TypeScript en todo el proyecto

- Tipar siempre props, estados y funciones.
- Ejemplo:

```
type Subject = { id: number; name: string; examDate: string; color: string };
const [subjects, setSubjects] = useState<Subject[]>([]);
```

2. Estructura de carpetas clara

- Separar los archivos por responsabilidad:
 - o **components/**: Componentes reutilizables de UI.
 - o **services/**: Lógica de acceso a datos y APIs (por ejemplo, DatabaseService).
 - o **hooks/**: Hooks personalizados (**useAuth**, **useDatabase**).
 - o **context/**: Contextos globales (**authContext**).
- Ejemplo de estructura:

```
src/
  components/
  services/
  hooks/
  context/
```

3. Nomenclatura consistente

- Componentes y archivos de componentes en PascalCase: **Dashboard.tsx**, **MateriaCard.tsx**.
- Hooks en camelCase y con prefijo **use**: **useAuth.ts**, **useDatabase.ts**.
- Variables y funciones descriptivas y en inglés o español, pero nunca abreviadas sin sentido.

4. Separar lógica de UI y lógica de negocio

- La lógica de negocio (acceso a datos, procesamiento, etc.) debe ir en servicios o hooks, no en los componentes.
- Los componentes deben enfocarse en la presentación y la interacción.
- Ejemplo:
 - o Lógica de autenticación en **AuthService** o **useAuth**.
 - o Lógica de base de datos en **DatabaseService** o **useDatabase**.

5. Comentarios claros y útiles

- Comentar solo lo necesario: explicar "por qué" se hace algo, no "qué" hace el código (si ya es obvio).

- Ejemplo:

```
// Sincroniza el usuario con Firestore al autenticarse
useEffect(() => { ... }, []);
```

6. Evitar lógica compleja en los componentes

- Si un componente crece mucho, extraer partes a funciones, hooks o componentes hijos.
- Mantener los componentes lo más simples y legibles posible.

7. Uso de control de versiones y ramas

- Trabajar en ramas feature/ o fix/ y hacer pull requests a develop.
- Escribir mensajes de commit claros y descriptivos.

Esta guía es un punto de partida. Para dudas, consulta la documentación oficial de React, TypeScript y el repositorio de FocuseAr.